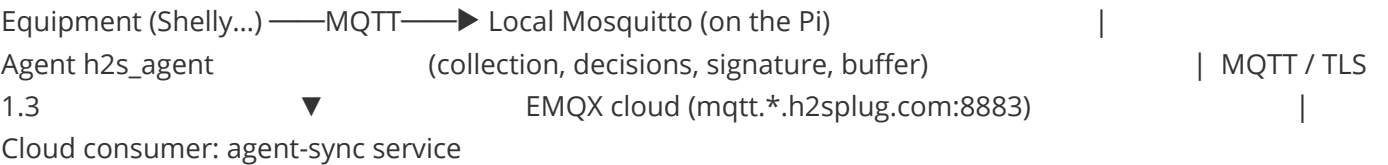


DS Onboarding — MQTT agent ↔ cloud channel (dev)

Guide for the DS (Raspberry Pi). The full channel reference is `docs/MQTT.md`; (cf. Gildas) this document is the operator's manual: how to connect, what is frozen, what remains open.

1. Two brokers, two roles



- **Local Mosquitto embedded on the Pi** — site equipment side (spec 18 §4.1.3). This is your field: `edge/deploy/mosquitto.conf` + `mosquitto.acl` (examples, to be validated on the actual device).
⚠ the example advertises `tls_version tlsv1.2`: the spec requires TLS 1.3 — must be aligned.
- **EMQX on the cloud side** — imposed by the spec §3.3, proven in lot 5 (tested ACLs, TLS 1.3). You don't need to master it: it's standard MQTT 5, `mosquitto_pub/sub` work on it for your tests.
- **No broker↔broker bridge**: the agent (`h2s_agent`) carries the flow between the two.

2. Dev access

	Value
Host	<code>mqtt.dev.h2splug.com</code>
Port	<code>8883</code> — TLS 1.3 only (TLS 1.2 is rejected)
Certificate	Let's Encrypt (standard ISRG Root X1 chain — the system trust store is enough, <code>ca_file</code> optional)
Account	<code>uuid:password</code> — provided by devops, one account per enrolled device
Client ID	the same <code>uuid</code> as the account

The account `uuid` is the agent identifier (`edge_clients.agent_uuid`) returned at enrollment. Password in a `0600` file (`password_file`), never in `agent.json`.

Ports 1883 (cleartext) and 18083 (console) = `127.0.0.1` only — invisible from the outside.

3. Topic layout — frozen, to adopt

The `energy/<device_id>/...` layout from the edge spec is **not** retained: the space is partitioned per agent (the ACL relies on it). An agent only exists under `h2s/agents/<its-uuid>/...`.

Topic	Direction	QoS	Envelope type
<code>h2s/agents/<uuid>/telemetry</code>	agent → cloud	1	<code>live</code> (site snapshot ~5 s) · <code>raw</code> (raw readings)
<code>h2s/agents/<uuid>/comptes-rendus</code>	agent → cloud	1	<code>records</code> · <code>command_result</code>
<code>h2s/agents/<uuid>/etat</code>	agent → cloud	1	<code>state</code> (version, buffer, clock, TPM probe)
<code>h2s/agents/<uuid>/ordres</code>	cloud → agent	1	<code>decision</code> · <code>command</code> — signed by the cloud

- `retain = false` everywhere, no last will (a testament can carry neither nonce nor fresh timestamp → rejected by the cloud).
- Persistent sessions (`clean_session = false`), stable identifiers.

4. Access rules — an agent only sees itself

`mqtt/acl.conf` (deny by default), proven by 11 cases on real EMQX:

Action	Verdict
Publish to its own <code>telemetry</code> / <code>comptes-rendus</code> / <code>etat</code>	✓
Subscribe to its own <code>ordres</code>	✓
Publish/subscribe to another agent's subtree	✗ (SUBACK 128 / drop)
Subscribe to <code>h2s/agents/#</code> , <code>#</code> , <code>\$SYS/#</code>	✗
Publish to its own <code>ordres</code>	✗ — that would mean forging a control order

No anonymous connection: an undeclared account cannot connect.

5. Envelope and signature — agent → cloud

```
{ "v": 1, "agent": "<uuid>", "ts": 1758000000, "nonce": "<32 hex>", "payload": "<exact JSON string>", "sig": "<base64url>" }
```

`sig` = Ed25519 signature (or ECDSA P-256 if the TPM supports it) of the canonical string `H2S-AGENT-V1` (D249), method `MQTT`, with the topic in place of the path:

```
H2S-AGENT-V1MQTT

## 


```

`payload` is an **exact JSON string** (not a nested object): it travels and is signed as-is — neither the broker nor a parser can touch a single byte of it.

Cloud-side checks: timestamp within ±5 min · known and non-revoked agent · verified signature · nonce never seen (`edge_nonces`, purged hourly) · topic belonging to the agent · envelope ≤ 256 KB · ≤ 5,000 messages/hour/agent.

Clock: the ± 5 min window requires reliable NTP on the Pi (chrony recommended) — otherwise serial `perime` rejections.

6. Orders — cloud → agent

```
{ "v": 1, "type": "decision", "payload": "<exact JSON string>", "signature": "<base64url>", "key_id": "<orders key>" }
```

Same signed payload as HTTP polling: signature of the cloud orders key, bound to the agent's site, coherent validity, known action, SEC1-6 rules applied identically. The transport decides nothing.

Idempotence: an order delivered twice (QoS 1 or fallback) is executed only once (`decision_cache` / `decision_id`; `edge_messages.message_id` for records).

7. Local Mosquitto (Pi) — your scope

`edge/deploy/` ships: `mosquitto.conf` (8883 TLS on the equipment LAN + 1883 loopback agent), `mosquitto.acl`, `agent.example.json`, `h2s-agent.service` (hardened systemd). To do:

1. Validate the config on the real Pi (marked "not verified on device");
2. Switch `tls_version` to 1.3;
3. Equipment → your local broker; agent `h2s_agent` → cloud EMQX.

`agent.json` (`cloud_mqtt` excerpt):

```
"cloud_mqtt": {  "host": "mqtt.dev.h2splug.com",  "port": 8883,  "tls": true,  "ca_file": "/etc/h2s-agent/ca-courtier.pem",  "password_file": "/etc/h2s-agent/mqtt.secret" }
```

Without `host`, the channel is not opened (previous behavior, HTTP polling only).

8. Answers to the devops questionnaire (point by point)

#	Question	Answer
1	FQDN DEV/TEST/PROD	<code>mqtt.dev.h2splug.com</code> (active); <code>mqtt.test.</code> / <code>mqtt.</code> reserved in DNS, envs not yet provisioned
2	8883 mandatory	Yes — TLS 1.3 only, verified
3	CA + client credential	Standard LE + username/password (<code>uuid:secret</code>). No mTLS today — if required, to be decided on the infra side
4	Client-ID	enrollment <code>uuid</code> (= account)
5	Topics + ACL	<code>h2s/agents/<uuid>/...</code> frozen, per-agent ACL (§3-4)
6	Downstream consumer	<code>agent-sync</code> (client <code>h2s-agent-sync</code>) — not yet deployed in dev, see §9
7	QoS / keepalive / retained	QoS 1 everywhere · persistent session · <code>retain=false</code> · no LWT
8	Max payload size	Envelope $\leq$ 256 KB cloud-side; EMQX <code>max_packet_size</code> = default 1 MiB (to calibrate if payloads > 256 KB — today the cloud limit governs)
9	Schema versioning	<code>v:1</code> + discriminated <code>type:</code> ; any evolution = new <code>v</code>
10	Key lifecycle	Enrollment by code (agent), MQTT accounts via Ansible <code>mqtt</code> role (prod) / <code>H2S_MQTT_AGENTS</code> (dev); revocation = account removal + <code>edge_clients</code> revoked
11	Public key registry	<code>edge_clients</code> (enrolled Ed25519/ECDSA-TPM key), verification in <code>AgentAuthenticator</code>
12	Anti-replay window	unique nonce + ± 5 min (<code>edge_nonces</code>)
13	Offline buffer	<code>h2s_agent/buffer.py</code> persistent on the Pi — size/expiry to be specified on the agent side
14	Metrics	<code>h2s_mqtt_*</code> Prometheus (backend :9000) + <code>GET /api/agent/status</code> on the agent; EMQX metrics to wire (TODO)
15	Network	Outbound only: TCP 8883 $\rightarrow$ <code>mqtt.dev.h2splug.com</code> (native TLS, no HTTP proxy). No inbound flow to the Pi required

9. Current dev state — worth knowing

-  Broker reachable, TLS 1.3, auth + ACL proven live on 2026-09-23.
-  One agent account provisioned (uuid sent separately).

-  `agent-sync` (the consumer) is **not deployed in dev**: your messages are accepted by the broker but nothing consumes them yet — signature/envelope cannot be verified end-to-end until the service runs. Signature verification will happen on the first microservices deployment or in a local test with the backend team.
-  `mqtt-init` only imports accounts on an empty base: adding an agent = extend `H2S_MQTT_AGENTS` (CI var `DEV_H2S_MQTT_AGENTS`) then `docker compose up -d --force-recreate mqtt-init emqx` on the VPS.

10. First connection checklist

1. `openssl s_client -connect mqtt.dev.h2splug.com:8883 -servername mqtt.dev.h2splug.com`
→ LE cert OK, TLS 1.3.
2. `mosquitto_sub -h mqtt.dev.h2splug.com -p 8883 --tls-version tlsv1.3 -u <uuid> -P <secret> -i <uuid> -t 'h2s/agents/<uuid>/ordres' -q 1` → SUBACK granted.
3. `mosquitto_sub ... -t 'h2s/agents/#'` → expected refusal (128).
4. `mosquitto_pub ... -t 'h2s/agents/<uuid>/telemetrie' -m '<signed envelope>'` → accepted.
5. `mosquitto_pub ... -t 'h2s/agents/<uuid>/ordres' -m 'x'` → refused (drop, `AUTHZ` log on broker side).